

MAGLALANG, SEAN GREI I.

C204 BSCS

700P

Finals Lab Task 1. Encapsulation

A Car That Works

For this program, you are tasked to define the following:

Class - Car:

- Properties:
 - `color` (type: str): Represents the color of the car.
 - `price` (type: float): Holds the price of the car.
 - `size` (type: str): Indicates the size of the car, where 'S' represents small, 'M' represents medium, and 'L' represents large.
- Constructor:
 - `__init__(self, color: str, price: float, size: str)`: Initializes the car's `color`, `price`, and `size` properties. The `size` is standardized to uppercase using `size.upper()`.
- Methods
 - Getter Methods:
 - `get_color(self) -> str`: Returns the car's color.
 - `get_price(self) -> float`: Returns the car's price.
 - `get_size(self) -> str`: Returns the car's size.
 - Setter Methods:
 - `set_color(self, color: str) -> None`: Sets the car's color to the specified value.
 - `set_price(self, price: float) -> None`: Sets the car's price to the specified value.
 - `set_size(self, size: str) -> None`: Sets the car's size to the specified value. The size should be one of 'S' for small, 'M' for medium, or 'L' for large. Use conversion of lowercase characters to uppercase using `size.upper()`.
 - `__str__` Method:
 - `__str__(self) -> str`: Returns a formatted string representing the car, following the format "Car (color) - P(price, formatted to two decimal places) - (size descriptor)". The size descriptor is determined based on the size character ('small' for 'S', 'medium' for 'M', and 'large' for 'L').
 - Example Strings:
 - For a red car priced at 19999.85 and of medium size: "Car (red) - P19999.85 - medium"
 - For a blue car priced at 50000.00 and large: "Car (blue) - P50000.00 - large"

Note: Each class should be defined in its own file, with the file name following camelCase conventions (e.g., `bankAccount.py`).

Source Code:

```
class Car:

    def __init__(self, color: str, price: float, size: str):

        self.__color = color
        self.__price = price
        self.__size = size.upper()

    def get_color(self) -> str:
        return self.__color

    def get_price(self) -> float:
        return self.__price

    def get_size(self) -> str:
        return self.__size

    def set_color(self, color: str) -> None:
        self.__color = color

    def set_price(self, price: float) -> None:
        self.__price = price

    def set_size(self, size: str) -> None:

        self.__size = size.upper()

    def __str__(self) -> str:

        if self.__size == 'S':
            size_desc = "small"
        elif self.__size == 'M':
            size_desc = "medium"
        elif self.__size == 'L':
            size_desc = "large"
        else:
            size_desc = "unknown"

        return f"Car ({self.__color}) - P{self.__price:,.2f} - {size_desc}"

if __name__ == "__main__":
    print('Action: Invoking the Car class constructor using Car("red",
19999.85, "M").')
    car1 = Car("red", 19999.85, "M")
    print("Output:")
    print(car1)
    print()

    print('Action: Invoking the Car class constructor using Car("blue",
```

```
50000.00, "L").')
    car2 = Car("blue", 50000.00, "L")
    print("Output:")
    print(car2)
    print()

    print('Action: Invoking the Car class constructor using Car("green",
12345.67, "S").')
    car3 = Car("green", 12345.67, "S")
    print("Output:")
    print(car3)
```

Sample Output:

```
Action: Invoking the Car class constructor using Car("red", 19999.85, "M").
Output:
Car (red) - P19,999.85 - medium

Action: Invoking the Car class constructor using Car("blue", 50000.00, "L").
Output:
Car (blue) - P50,000.00 - large

Action: Invoking the Car class constructor using Car("green", 12345.67, "S").
Output:
Car (green) - P12,345.67 - small

Process finished with exit code 0
```